

### Q59. Linear Search – Search in Mixed Dataset

Given the unsorted dataset [8, 19, 27, 44, 12, 31, 50], apply Linear Search to find the element 12. Clearly interpret the problem and explain the search process step-by-step. Count the number of comparisons made until the element is found. Analyze the best-case, average-case, and worst-case time complexity. Verify the correctness of the result. Interpret the efficiency of Linear Search for moderate-sized unsorted lists.

### IMPLEMENTATION:

```
File Edit Format Run Options Window Help
n = int(input("Enter the number of elements: "))
data = []
print("Enter the elements:")
for i in range(n):
    data.append(int(input()))
target = int(input("Enter the element to search: "))
comparisons = 0
found = False
for i in range(n):
    comparisons += 1
    if data[i] == target:
        print("Element found at index:", i)
        print("Position:", i + 1)
        print("Number of comparisons:", comparisons)
        found = True
        break
if not found:
    print("Element not found")
    print("Number of comparisons:", comparisons)
```

### OUTPUT:

```
===== RESTART: D:/DAA pratical/co-2(1).py
Enter the number of elements: 7
Enter the elements:
8
19
27
44
12
31
50
Enter the element to search: 12
Element found at index: 4
Position: 5
Number of comparisons: 5
>
```

#### Q41. Closest Pair – Transportation Hub Coordinates

A set of transport hubs is represented by coordinates  $\{(1,7), (4,9), (6,8), (3,5), (8,2), (2,6)\}$ . Apply the Closest Pair Brute Force algorithm to identify the nearest hubs. Clearly interpret the problem and justify the chosen method. Compute the distance between all possible pairs and show the calculations clearly. Identify the closest pair and count the number of pairwise comparisons performed. Analyze the time complexity of the brute force approach. Verify the correctness of the result and interpret its importance in transportation network planning.

#### IMPLEMENTATION:

```
import math

n = int(input("Enter the number of points: "))

points = []

print("Enter the coordinates (x y):")
for i in range(n):
    x, y = map(int, input().split())
    points.append((x, y))

min_dist = float('inf')
closest_pair = ()
comparisons = 0

print("\nDistances between all pairs:")

for i in range(n):
    for j in range(i + 1, n):
        comparisons += 1
        d = math.sqrt((points[i][0] - points[j][0])**2 +
                      (points[i][1] - points[j][1])**2)

        print(points[i], "-", points[j], "=", round(d, 2))

        if d < min_dist:
            min_dist = d
            closest_pair = (points[i], points[j])

print("\nClosest Pair:", closest_pair)
print("Minimum Distance:", round(min_dist, 2))
print("Number of Comparisons:", comparisons)
```

#### OUTPUT:

```
===== RESTART: D:/DAA practical/c0-2(2)
Enter the number of points: 6
Enter the coordinates (x y):
1 7
4 9
6 8
3 5
8 2
2 6

Distances between all pairs:
(1, 7) - (4, 9) = 3.61
(1, 7) - (6, 8) = 5.1
(1, 7) - (3, 5) = 2.83
(1, 7) - (8, 2) = 8.6
(1, 7) - (2, 6) = 1.41
(4, 9) - (6, 8) = 2.24
(4, 9) - (3, 5) = 4.12
(4, 9) - (8, 2) = 8.06
(4, 9) - (2, 6) = 3.61
(6, 8) - (3, 5) = 4.24
(6, 8) - (8, 2) = 6.32
(6, 8) - (2, 6) = 4.47
(3, 5) - (8, 2) = 5.83
(3, 5) - (2, 6) = 1.41
(8, 2) - (2, 6) = 7.21

Closest Pair: ((1, 7), (2, 6))
Minimum Distance: 1.41
Number of Comparisons: 15
>>|
```

### Q131. Selection Sort – Dataset with Alternating Extremes

Apply the Brute Force technique using Selection Sort on the dataset [91, 3, 78, 5, 64, 7]. Clearly interpret the problem and justify why Selection Sort is suitable for repeatedly selecting the minimum element from the unsorted portion. Perform the sorting step-by-step and show the intermediate array after each pass. Count the total number of comparisons and swaps made during execution. Analyze the time complexity in best, average, and worst cases. Verify the correctness of the final sorted output and interpret the efficiency of the algorithm on this dataset.

#### IMPLEMENTATION:

```
File Edit Format Run Options Window Help
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
comparisons = 0
swaps = 0
print("\nSorting Process:")
for i in range(n - 1):
    min_index = i
    for j in range(i + 1, n):
        comparisons += 1
        if arr[j] < arr[min_index]:
            min_index = j
    if min_index != i:
        arr[i], arr[min_index] = arr[min_index], arr[i]
        swaps += 1
    print("Pass", i + 1, ":", arr)
print("\nSorted Array:", arr)
print("Total Comparisons:", comparisons)
print("Total Swaps:", swaps)
```

#### OUTPUT:

```
Enter the number of elements: 6
Enter the elements:
91
3
78
5
64
7

Sorting Process:
Pass 1 : [3, 91, 78, 5, 64, 7]
Pass 2 : [3, 5, 78, 91, 64, 7]
Pass 3 : [3, 5, 7, 91, 64, 78]
Pass 4 : [3, 5, 7, 64, 91, 78]
Pass 5 : [3, 5, 7, 64, 78, 91]

Sorted Array: [3, 5, 7, 64, 78, 91]
Total Comparisons: 15
Total Swaps: 5
```